CS 301
High-Performance Computing

Assignment 03 - Serial Interpolation on Scattered Data

Jeet Modi (202301412)
Aaditya Thakkar (202301417)

February 18, 2026

Contents

1 Introduction **3**

2 Problem Description **3**

3 Interpolation Method **3**
3.1 Cell Identification 3
3.2 Local Coordinates 3
3.3 Bilinear Weights 3

4 Implementation Approach **4**
4.1 Data Structures 4
4.2 Algorithm Workflow 4

5 Time Complexity Analysis **4**

6 Arithmetic Operations Per Particle **4**

7 Memory Access Analysis **5**

8 Arithmetic Intensity **6**

9 Performance Results **6**
9.1 Execution Time Table 6
9.2 Execution Time Plot 7

10 Discussion **7**

11 Conclusion **7**

1 Introduction

This assignment studies the implementation and performance characteristics of a serial bilinear interpolation algorithm that maps scattered particle data onto a structured grid. The focus is on computational cost, arithmetic workload, memory-access behavior, and runtime performance.

2 Problem Description

Given scattered points

$$S = \{(x_i, y_i, f_i)\}$$

with $f_i = 1$, the objective is to accumulate their values onto a structured grid

$$G = \{(X_i, Y_j)\}$$

with spacing

$$\Delta x = \frac{1}{NX}, \quad \Delta y = \frac{1}{NY}$$

Each particle contributes to the four surrounding grid nodes.

3 Interpolation Method

For a particle at (x, y):

3.1 Cell Identification

$$i = \left\lfloor \frac{x}{\Delta x} \right\rfloor, \quad j = \left\lfloor \frac{y}{\Delta y} \right\rfloor$$

3.2 Local Coordinates

$$l_x = x - X_i, \quad l_y = y - Y_j$$

3.3 Bilinear Weights

$$w_{i,j} = \frac{(\Delta x - l_x)(\Delta y - l_y)}{\Delta x \Delta y}$$

$$w_{i+1,j} = \frac{l_x(\Delta y - l_y)}{\Delta x \Delta y}$$

$$w_{i,j+1} = \frac{(\Delta x - l_x) l_y}{\Delta x \Delta y}$$

$$w_{i+1,j+1} = \frac{l_x l_y}{\Delta x \Delta y}$$

4 Implementation Approach

4.1 Data Structures

- 1D array for mesh (row-major layout)
- Structure array for particle coordinates

Mesh indexing:

$$\text{index} = j \times GRID_X + i$$

4.2 Algorithm Workflow

1. Read input
2. Initialize mesh
3. Loop over particles
4. Compute indices and weights
5. Update four grid nodes
6. Repeat for $Maxiter$

5 Time Complexity Analysis

Each particle requires a constant number of operations. Thus,

$$T(N) = \mathcal{O}(N)$$

For $Maxiter$ iterations:

$$T_{total} = \mathcal{O}(N \times Maxiter)$$

6 Arithmetic Operations Per Particle

Per particle:

- 2 divisions (index scaling)
- 2 floor operations
- 2 subtractions (local coordinates)
- 8 multiplications (weight computation)
- 4 additions (mesh accumulation)

Total $\approx$ 18–22 FLOPs per particle.

7 Memory Access Analysis

For each particle, memory activity consists of:

- 2 reads for particle coordinates (x, y)
- 4 read-modify-write operations for grid nodes

Each grid update involves:

$$\text{Load } G_{ij} \rightarrow \text{Add weighted value} \rightarrow \text{Store } G_{ij}$$

Thus per particle:

$$\text{Memory reads} = 2 \text{ (particles)} + 4 \text{ (grid loads)} = 6$$

$$\text{Memory writes} = 4 \text{ (grid stores)}$$

Total memory operations per particle:

$$M = 10$$

Now counting arithmetic precisely:
Index computation:

$$2 \text{ divisions} + 2 \text{ floor}$$

Local coordinate computation:

$$2 \text{ subtractions}$$

Weight computation: Each weight requires:

$$2 \text{ subtractions} + 2 \text{ multiplications} + 1 \text{ division}$$

For four weights:

$$8 \text{ multiplications} + 4 \text{ divisions} + 8 \text{ subtractions}$$

Mesh accumulation:

$$4 \text{ multiplications} + 4 \text{ additions}$$

Total arithmetic operations:

$$A \approx 24\text{–}28 \text{ FLOPs}$$

Arithmetic intensity becomes:

$$AI = \frac{A}{M} \approx \frac{26}{10} \approx 2.6$$

Since arithmetic intensity is low and memory operations are relatively high, performance is dominated by memory bandwidth rather than computation.

Furthermore, particle positions are randomly distributed. Therefore:

- Spatial locality is weak
- Cache-line reuse is limited
- Grid updates often miss L1/L2 cache

As grid size increases, cache capacity is exceeded, and most accesses are served from higher cache levels or main memory, making the algorithm memory-bound.

8 Arithmetic Intensity

$$\text{Arithmetic Intensity} \approx 2.6$$

This confirms the memory-bound behavior of the kernel.

9 Performance Results

9.1 Execution Time Table

Case	Lab PC (s)	Cluster (s)	Speedup (Lab/Cluster)
(a)	0.084440	0.260000	0.325
(b)	0.470347	1.150000	0.409
(c)	0.347945	0.930000	0.374
(d)	1.896351	4.720000	0.402
(e)	1.874231	3.280000	0.571

Table 1: Execution time comparison between Lab PC and HPC Cluster

9.2 Execution Time Plot

Figure 1: Execution time comparison between Lab PC and HPC Cluster

10 Discussion

Speedup values below 1 indicate that the Lab PC outperforms the HPC cluster for serial execution. Since the implementation does not exploit parallelism, single-core frequency and memory latency dominate performance.

11 Conclusion

The serial bilinear interpolation algorithm exhibits linear time complexity and low arithmetic intensity, making it primarily memory-bound.

Performance results show that the Lab PC outperforms the multi-core HPC cluster for all cases. Since the implementation is purely serial, it uses only one core on the cluster node. HPC processors typically run at lower per-core clock frequencies, optimized for parallel workloads.

Therefore, the higher single-core frequency and lower effective memory latency of the Lab PC lead to better performance for this memory-bound serial application.